VFI Toolkit: Pseudocodes

Robert Kirkby*

October 27, 2026

*Kirkby: Victoria University of Wellington. Please address all correspondence about this article to Robert Kirkby at <robertdkirkby@gmail.com>, robertdkirkby.com, vfitoolkit.com.

Contents

1 Introduction **1**

2 Value Function Iteration: Solving for V and Policy **1**
2.1 Infinite Horizon: decision variable, markov shock . . . 1
2.1.1 Howards improvement: iterated and greedy . . . 2
2.1.2 Relative VFI and Endogenous VFI . . . 4
2.1.3 Refinement . . . 5
2.2 Finite Horizon: decision variable, markov shock . . . 6
2.2.1 Finite Horizon: semi-exogenous state . . . 6
2.2.2 Finite Horizon: experienceasset . . . 7
2.2.3 How interpolation is done . . . 8
2.2.4 Finite Horizon: riskyasset . . . 9
2.2.5 Finite Horizon: standard & riskyasset . . . 10
2.2.6 Finite Horizon: Epstein-Zin preferences . . . 11

3 Agent Distribution **14**
3.1 Stationary Agent Distribution in Infinite Horizon . . . 14
3.1.1 Stationary Agent Distribution as Eigenvector . . . 15
3.1.2 Simulate Agent Distribution . . . 15
3.1.3 Iterate Agent Distribution . . . 16
3.1.4 Tan Improvement to Iterating Agent Distribution . . . 17
3.2 Agent Distribution in Finite Horizon . . . 18
3.2.1 Simulating the Agent Distribution . . . 18
3.2.2 Iterating the Agent Distribution . . . 19
3.2.3 Tan Improvement Iterating the Agent Distribution . . . 19
3.2.4 Tan Improvement with Semi-Exogenous shocks . . . 20
3.2.5 Iterating the Agent Distribution with 'nProbs' . . . 21

4 Stationary General Equilibrium **23**

5 Transition Path General Equilibrium **24**

1 Introduction

This document presents pseudocodes for various commands from VFI Toolkit. The actual implementations in VFI Toolkit often involve parallelized versions of the pseudocodes presented here, and also typically have options to use more-or-less parallelization (versus looping).

This document is a work-in-progress. Please request pseudocode for any specific command you want to see it for (either by email or on discourse.vfitoolkit.com).

Several features of VFI Toolkit have their own companion write-ups that go into more detail and include figures. Two that are relevant to the algorithms described here are Divide and Conquer (github.com/robertdkirkby/DivideAndConquer) and the Grid Interpolation Layer (github.com/robertdkirkby/GridInterpolationLayer); each github repository contains the corresponding pdf.

Throughout I use the standard notation of VFI Toolkit: so d is a decision variable, a is endogenous state (and a' the next-period endogenous state), z is a markov exogenous state, e is an i.i.d. exogenous state, u is an i.i.d. shock that occurs between periods. The transition probability from current exogenous state z to next-period exogenous state z' is denoted by $\pi_z(z, z')$. V is the value function. For finite horizon problems there are N_j periods, indexed by j.

Throughout, $D(\cdot)$ denotes the set of feasible choices conditional on the current states.

In many pseudocodes $g(a, z)$ denotes a policy, with $g^d(a, z)$ being the policy for the decision variables and $g^{a'}(a, z)$ being the policy for the next-period endogenous state.

2 Value Function Iteration: Solving for V and Policy

2.1 Infinite Horizon: decision variable, markov shock

Starting from an initial guess for the value function, we iterate on the value function until convergence. To speed things up Howards improvement is used (Section 2.1.1), and when the discount factor is high, endogenous VFI is used (Section 2.1.2).

Declare initial value V_0.
Declare iteration count $n = 0$.
while $||V_n - V_{n-1}|| >$ 'tolerance constant'
  Increment n. Let $V_{old} = V_{n-1}$.
  for A ll values z
    Calculate $E[V_{old}(a', z')|z] = \sum_{z'} V_{old}(a', z')\pi_z(z, z')$
    for A ll values of a
      Calculate $V_n(a, z) = max_{d,a' \in D(a,z)} F(d, a', a, z) + \beta E[V_{old}(a', z')]$

... and keep the $argmax$ $g_n(a,z)$.

end for

end for

if U seHowards==1

for n Howards number of times

Update $V_n(a,z) = F(g_n^d(a,z), g_n^{a'}(a,z), a, z) + \beta E[V_n(g_n^{a'}(a,z), z')]$

end for

end if

end while

return V_n, g_n

Note that by making the outer-loop over z we are able to reduce the number of times we compute the expectations (which depend on z and not on a).

2.1.1 Howards improvement: iterated and greedy

Write $F_g(a,z) = F(g^d(a,z), g^{a'}(a,z), a, z)$ for the return under policy g, and P_g for the transition matrix on (a,z) that policy g implies, so $(P_g V)(a,z) = \sum_{z'} V(g^{a'}(a,z), z')\pi_z(z,z')$. The value of following policy g forever is the solution to $V = F_g + \beta P_g V$. There are two versions of Howards, set by *vfoptions.howardsgreedy*.[1]

Iterated Howards (*vfoptions.howardsgreedy=0*, the default; a.k.a. modified policy function iteration) applies the policy's Bellman equation a fixed number of times,

for $nHowards$ number of times

Update $V_n(a,z) = F_{g_n}(a,z) + \beta \sum_{z'} V_n(g_n^{a'}(a,z), z')\pi_z(z,z')$

end for

$nHowards$ is *vfoptions.howards*. Its default depends on the discount factor, as the best number of Howards steps grows roughly like $1/(1-\beta)$ (except when endogenous VFI is being used, Section 2.1.2, where the best number barely depends on β). The defaults come from runtimes in the VFI Toolkit test bank (*CoreInfHorzVFIAlgoTests*). If the discount factor depends on the exogenous state, the default uses its largest value. There are two implementations of the same update, set by *vfoptions.howardssparse*: one indexes V_n at $g_n^{a'}(a,z)$, the other builds P_{g_n} as a sparse matrix once and then does $nHowards$ sparse matrix-vector products. They give the same answer. The sparse version is faster when there are many exogenous states, and with the grid interpolation layer.

Greedy Howards (*vfoptions.howardsgreedy=1*; a.k.a. policy function iteration) instead solves for the value of following g_n forever, which is the limit of iterated Howards as $nHowards \to \infty$,

[1] Rendahl (2022), Bakota and Kredler (2022) and Phelan and Eslami (2022) are relevant literature that discusses these ideas in more depth.

Construct the sparse matrix P_{g_n} and the vector F_{g_n}
Solve the linear system $(I - \beta P_{g_n})V_n = F_{g_n}$ ▷ a sparse direct solve, $V_n = (I - \beta P_{g_n})^{-1} F_{g_n}$

Each greedy step costs much more than an iterated Howards step, but far fewer iterations are needed. Greedy Howards is implemented without a d variable; with a d variable it works through the refinement (Section 2.1.3), which removes d before iterating.

If some states have $V = -\infty$ (e.g., no feasible choice), the linear solve has to be done on the 'live' states only (in floating point, a single $-\infty$ in F_{g_n} makes the solution NaN everywhere). A state is 'dead' if $V_n = -\infty$ there, or if its policy puts positive probability on a dead state (repeated until no new states are added). Ordering the states live-then-dead makes $(I - \beta P_{g_n})$ block triangular, so we solve the live block on its own and set $V_n = -\infty$ on the dead states.

Howards improvement algorithm is to use the current policy to evaluate/update the value function a couple of times. This is faster as it skips the maximization step (which is computationally the most costly) and because the policy function typically is 'pointing' in the direction of the solution, so a few cheap computations to move further in this direction is worthwhile. $UseHowards$ is the conditions under which Howards improvement algorithm is used, and is the same for iterated and greedy Howards: Howards is used in iteration n when $||V_n - V_{n-1}||$ is finite, $||V_n - V_{n-1}|| > 10\times$*vfoptions.tolerance*, and $n <$*vfoptions.maxhowards*. So we skip Howards improvement algorithm in the first iteration (where $||V_n - V_{n-1}||$ is still infinite), stop using it after *vfoptions.maxhowards* iterations (default 500; just so it cannot cause convergence to fail if things are going wrong), and also stop using Howards improvement algorithm once we are within one order of magnitude of convergence of the value function.[2]

Because Howards is switched off once we are within one order of magnitude of convergence, the last iteration is always a plain value function iteration step, without Howards.

To summarise the current defaults: iterated Howards (*vfoptions.howardsgreedy=0*); *vfoptions.howards=round*$(2/(1-\beta))$ kept between 20 and 640, or 40 (80 with sparse Howards) when endogenous VFI is used; *vfoptions.howardssparse=1* with the grid interpolation layer (other than *vfoptions.preGI=1*, for which sparse is not implemented) and when $N_z \geq 50$ and $N_a \geq 500$, and 0 otherwise; *vfoptions.maxhowards=500*; *vfoptions.tolerance*$= 10^{-9}$; and *vfoptions.maxiter*$= 10^4$.

[2]This is because otherwise Howards leads to very small errors. You won't notice them normally, but when trying to compute a placebo transition path between stationary equilibria they are problematic as the transition won't use Howards and so tries to go to a very slightly different place. If you are thinking 'but I saw the proof in Sargent & Stachurski that Howards is fine and gives same answer' the trick is that the last line of their proof assumes that Howards evaluates the 'policy greedy' value function, but in practice you will never do this. (Placebo transition=a transition path in which nothing changes, this is actually quite a demanding test of accuracy; this is not a dig at S&S, all textbooks do the same.)

2.1.2 Relative VFI and Endogenous VFI

Value function iteration converges in levels at rate β, so when β is close to one it is slow, however many Howards steps we take. But much of the error is in a part of V that does not matter for the policy. Relative VFI (Bray, 2019) and endogenous VFI (Bray, 2017) exploit this. Both are set by *vfoptions.relativevfi=1*; relative VFI is just the name for the case without z.

The key observation is that if we add to V some function of the exogenous state, $c(z)$, then the Bellman operator T shifts the result by a function of z only, and so the policy is unchanged,

$$T(V + c)(a, z) = TV(a, z) + \beta \sum_{z'} c(z')\pi_z(z, z').$$

So we only need to iterate until $V_n - V_{n-1}$ is constant across a for each z (rather than until it is zero), and we can then recover the level of V by solving a small linear system in z. The iterations are exactly the same as in plain value function iteration (including Howards); only the stopping rule and a final 'level recovery' step change.

Declare initial value V_0.
Declare iteration count $n = 0$.
while $max_z\,[max_a(V_n(a, z) - V_{n-1}(a, z)) - min_a(V_n(a, z) - V_{n-1}(a, z))] >$ 'tolerance constant'
  Increment n, and do one iteration of value function iteration (with Howards) exactly as above
end while
Calculate $\Delta(z) = mean_a[V_n(a, z) - V_{n-1}(a, z)]$ ▷ over the a at which this is finite
Solve $c = \Delta + \beta\pi_z c$ for $c(z)$, so $c = (I - \beta\pi_z)^{-1}\Delta$ ▷ an N_z-by-N_z linear system
Update $V_n(a, z) = V_n(a, z) + \beta \sum_{z'} c(z')\pi_z(z, z')$
return V_n, g_n

The stopping rule is the span (max minus min) over a of $V_n - V_{n-1}$, within each z. The level recovery works because the last iteration is a plain value function iteration step (Howards is switched off near convergence), so $V_n = TV_{n-1}$ and $V_n - V_{n-1} = \Delta(z)$ up to the tolerance; then $V_{n-1} + c$ is the fixed point of T, and $T(V_{n-1} + c) = V_n + \beta\pi_z c$. Without z (relative VFI), $c = \Delta/(1 - \beta)$ and the update is just $V_n = V_n + \beta\Delta/(1 - \beta)$.

Endogenous VFI does the same iterations as plain VFI, only with a different stopping rule, so its extra cost per iteration is just the span calculation (plus the final linear system, once). How much it saves depends on β: in the VFI Toolkit test bank the runtime of the best plain VFI relative to the best endogenous VFI (each with its best number of Howards steps) had a median of 0.96 at $\beta = 0.9$, 1.15 at $\beta = 0.95$, 1.4 at $\beta = 0.97$, 2.3 at $\beta = 0.99$ and 4 at $\beta = 0.995$. So the default is *vfoptions.relativevfi=1* when $\beta \geq 0.97$ (and 0 otherwise). With multiple discount factors, β here is their product (the largest value of each, if one depends on the exogenous state).

Endogenous VFI only works when $T(V + c) = TV + \beta\pi_z c$, which is not the case with Epstein-Zin preferences, ambiguity aversion, sophisticated quasi-hyperbolic discounting (naive quasi-hyperbolic

is fine, as its continuation value is a standard VFI), or endogenous exit (the exit value does not shift with V). It is currently implemented for pure discretization and refinement, with and without the grid interpolation layer and semi-exogenous states, and only on the GPU.

2.1.3 Refinement

If you set *vfoptions.solnmethod = 'purediscretization_refinement';* then the algorithm will be the 'refinement'. Refinement is based on the observation that d does not enter the expectations next period value function, only the return function. So we can 'pre-solve', choosing d to maximize the return function $F(d, aprime, a, z)$ to get $d^*(aprime, a, z)$, the decision that maximizes the return function given $(aprime, a, z)$, and associated maximum values $F^*(aprime, a, z)$. We then perform standard value function iteration, but just using $F^*(aprime, a, z)$, so we have essentially removed d from the problem on which we have to iterate. Once we finish solving we can put the optimal policy for $aprime$ into $d^*(aprime, a, z)$ to get the optimal policy for d. This is faster because we just pre-solve for d once, and thereby avoid having to solve for it as part of every iteration.

for A ll values z ▷ First, pre-solve for d
  for A ll values a
    for A ll values a'
      Calculate $F^*(a', a, z) = max_{d \in D(a,z)} F(d, a', a, z)$ and the argmax $d^*(a', a, z)$
    end for
  end for
end for
Declare initial value V_0. ▷ Now do standard VFI without d variable
Declare iteration count $n = 0$.
while $||V_n - V_{n-1}||$ > 'tolerance constant'
  Increment n. Let $V_{old} = V_{n-1}$.
  for A ll values z
    Calculate $E[V_{old}(a', z')|z] = \sum_{z'} V_{old}(a', z')\pi_z(z, z')$
    for A ll values of a
      Calculate $V_n(a, z) = max_{a' \in D(a,z)} F^*(a', a, z) + \beta E[V_{old}(a', z')]$
      ... and keep the $argmax\ g_n^{a'}(a, z)$.
    end for
  end for
  if U seHowards==1
    for n Howards number of times
      Update $V_n(a, z) = F^*(g_n^{a'}(a, z), a, z) + \beta E[V_n(g_n^{a'}(a, z), z')]$
    end for
  end if
end while

Evaluate $g_n^d(a,z) = d^*(g_n^{a'}(a,z),a,z)$ ▷ Recover the policy for d

return V_n, g_n

Note that the only difference between the refinement is internal. All inputs and outputs will be identical. Refinement also has the nice property that if we loop when calculating d^* and then parallize the value function iteration itself, we reduce the memory requirements as well as reducing the runtime making it possible to solve more complex problems (as long as they have a d variable). Obviously the refinement is not possible in models that do not have a d variable to begin with.[3]

2.2 Finite Horizon: decision variable, markov shock

The basic algorithm is simple backward iteration. Let Θ be the full set of model parameters, and let θ_j be the values of those parameters that are relevant in period j.[4]

Solve for final period, N_j: $V_{N_j}(a,z) = \max_{d,a' \in D(a,z)} F(d,a',a,z)$

... and keep the $\arg\max$ g_{N_j}.

for j count backward from $N_j - 1$ to 1

  Get θ_j from Θ

  for A ll values of z

    Calculate $E[V_{j+1}(a',z')|z] = \sum_{z'} V_{j+1}(a',z')\pi_z(z,z')$

    for A ll values of a

      Calculate $V_j(a,z) = \max_{d,a' \in D(a,z)} \{F(d,a',a,z) + \beta E[V_{j+1}(a',z')|z]\}$

      ... and keep the $\arg\max$ $g_j(a,z)$.

    end for

  end for

end for

return $[V_1, V_2, ..., V_{N_j}]$, $[g_1, g_2, ..., g_{N_j}]$

If you were writing custom code for a specific problem, you would likely try to exploit features such as monotonicity. However, because VFI Toolkit is designed to solve generic problems, it uses this simple and robust approach.

2.2.1 Finite Horizon: semi-exogenous state

We now add a semi-exogenous state. Semi-exogenous shocks are exogenous states for which the transition probabilities depend on a decision variable (see Life-Cycle Model 30 for an example). Thus, $\pi_{semiz}(semiz'|semiz,d)$ gives the probability of the next-period semi-exogenous state as a

[3] Pure-discretization makes this refinement easy, but in principle it should be possible to do with algorithms that use functional forms to fit the value and policy functions.

[4] So if a parameter is independent of age, then it appears unchanged in θ_j. If a parameter depends on age, then θ_j contains only the value relevant at age j.

function of the current semi-exogenous state and the decision variable.

The main difference is that the expectations term is now more complex.

Solve for final period, N_j: $V_{N_j}(a, semiz, z) = \max_{d,a' \in D(a,semiz,z)} F(d, a', a, semiz, z)$
... and keep the $\arg\max$ g_{N_j}.
for j count backward from $N_j - 1$ to 1
Get θ_j from Θ
for A ll values of d
for A ll values of $(semiz, z)$
Calculate $E[V_{j+1}(a', semiz', z')|d, semiz, z]$
$= \sum_{semiz',z'} V_{j+1}(a', semiz', z')\pi_z(z, z')\pi_{semiz}(semiz'|semiz, d)$
for A ll values of a
Calculate $V_j(a, semiz, z|d)$
$= \max_{a' \in D(a,semiz,z)} \{F(d, a', a, semiz, z) + \beta E[V_{j+1}(a', semiz', z')|d, semiz, z]\}$
... and keep the $\arg\max$ $g_j(a, semiz, z|d)$.
end for
end for
end for
$V_j(a, semiz, z) = \max_d V_j(a, semiz, z|d)$
Let $d^*(a, semiz, z) = \arg\max_d V_j(a, semiz, z|d)$.
Then $g_j(a, semiz, z) = g_j(a, semiz, z|d^*(a, semiz, z))$.
end for
return $[V_1, V_2, ..., V_{N_j}]$, $[g_1, g_2, ..., g_{N_j}]$

The $E[V_{j+1}(a', semiz', z')|d, semiz, z]$ term is doing a lot of work here. It adds an extra dimension to the expectations term, because the expectation must now be computed separately for each value of d and for each current pair $(semiz, z)$. For this reason, it is natural to solve the problem conditional on d first, and then maximize over d afterward. Here, values of d that are not in $D(a, semiz, z)$ are implicitly assigned a return of $-\infty$, so they will never be chosen.

2.2.2 Finite Horizon: experienceasset

'experienceasset' is an endogenous state where aprime cannot be chosen directly, but is instead a function of a decision variable and this period endogenous state; $aprime(d, a)$. The basic idea of how VFI Toolkit handles these is to evaluate $aprime(d_i, a_i)$ as a value on the grids of d and a, and then linearly interpolate this $aprime(d_i, a_i)$ onto the grid for a by allocating it to the grid-points above and below that value with linear weights.

So in a finite-horizon model with no other endogenous states, and no decision variables except those influencing the next period experienceasset, the algorithm is

Solve for final period, N_j: $V_{N_j}(a,z) = max_{d \in D(a,z)} F(d,a,z)$

... and keep the argmax g_{N_j}.

for j count backward from $N_j - 1$ to 1

Get θ_j from Θ

for A ll values z

Calculate $E[V_{j+1}(a',z')]$

Evaluate $aprime(d,a)$ on the grids for d and a

Interpolate $aprime(d,a)$ onto the grid for a; get $aprime_i(d,a)$ and $aProbs(d,a)$, the index for the lower grid point and the probability of the lower grid point.

Switch $E[V_{j+1}(a',z')]$ to $E[V_{j+1}(d,a,z')]$ using $aprime_i(d,a)$ and $aProbs(d,a)$ to replace a' with d and a.

for A ll values of (a,z)

Calculate $V_j(a,z) = max_{d \in D(a,z)} F(d,a,z) + \beta E[V_{j+1}(d,a,z')]$

... and keep the $argmax$ $g_j(a,z)$.

end for

end for

end for

return $[V_1, V_2, ..., V_{N_j}]$, $[g_1, g_2, ..., g_{N_j}]$

notice that a' no longer appears in the return function. The main trick is to replace $E[V_{j+1}(a',z')]$ to $E[V_{j+1}(d,a,z')]$, by interpolating $aprime(a,z)$ onto the grid on a.

The way interpolation is done is that for a value of $aprime$ that we want to interpolate onto the grid $[a_1, a_2, \ldots, a_n]$ we, (i) find the a_i that is the largest grid point that has a value less than (or equal to) $aprime$, call this the lower grid point, (ii) we allocate $aprime$ to both the lower grid point a_i and the upper grid point a_{i+1} using weights, (iii) we use linear weights for a_i and a_{i+1}, so if $aprime$ is halfway between the two we give a weight of 0.5 to each, this is implemented by giving the lower grid point, a_i a weight of $1-(aprime-a_i)/(a_{i+1}-a_i)$ (the weight for the upper grid point is just 1-this). This interpolation can be seen in CreateExperienceAssetFnMatrix_Case1.m, the first part of the code creates $aprime(a,z)$ and then the last 30-40ish lines are doing the interpolation onto the grid, note that we only need to keep the lower grid point index and the probability assigned to the lower grid point (as the upper grid point is just adding one to this index, and the probability is one minus this).

2.2.3 How interpolation is done

VFI Toolkit uses linear interpolation as part of alternative endogenous states like experienceasset, experienceassetu, riskyasset, and more. There are two halves to how the toolkit uses interpolation. One half is to put points onto a grid (as a lower-grid-point index and the interpolation weight on that lower grid point, which together implicitly determine the upper grid point and its weight).

The second half is to use this to 'switch' grids.

For example in the previous pseudocode we saw the first half as:
Interpolate $aprime(d,a)$ *onto the grid for* a*; get* $aprime_i(d,a)$ *and* $aProbs(d,a)$*, the index for the lower grid point and the interpolation weight on the lower grid point.*
The way that the toolkit interpolates a value onto the grid is to use linear interpolation. So say we have that the value we want to interpolate onto the grid is x, and the grid is $\{x_1, x_2, \ldots, x_n\}$. If $x \leq x_1$ we assign all the weight to x_1. If $x \geq x_n$ we assign all the weight to x_n. Otherwise, we find the two grid points either side of x, call them x_i and x_{i+1}, so that $x_i \leq x < x_{i+1}$. Then we call x_i the lower grid point, and assign it the interpolation weight $1 - \frac{(x-x_i)}{(x_{i+1}-x_i)}$ (which is between 0 and 1, and is 1 if $x = x_i$). The interpolation weight on the upper grid point is then $\frac{(x-x_i)}{(x_{i+1}-x_i)}$, which is also between 0 and 1 and approaches 1 as x approaches x_{i+1}. We only need to store the lower-grid-point index and the interpolation weight on the lower grid point, because the upper grid point is then just the next point on the grid and its weight is one minus the lower-grid-point weight.

While the second half in the previous pseudocode we saw as:
Switch $E[V_{j+1}(a',z')]$ *to* $E[V_{j+1}(d,a,z')]$ *using* $aprime_i(d,a)$ *and* $aProbs(d,a)$ *to replace* a' *with* d *and* a*.*
The way the toolkit does this is that for every (d,a) we have a value $aprime(d,a)$ which we associated in the first half with a lower-grid-point index and interpolation weight, $aprime_i(d,a)$ and $aProbs(d,a)$. Looking at $E[V_{j+1}(a',z')]$ and ignoring the $E[\cdot]$ for the moment, we get

$$\begin{aligned} V_{j+1}(a',z') &= aProbs(d,a)\, V_{j+1}(a_{aprime_i(d,a)}, z') \\ &\quad +(1 - aProbs(d,a))\, V_{j+1}(a_{aprime_i(d,a)+1}, z') \\ &\equiv V_{j+1}(d,a,z'). \end{aligned}$$

At the lower and upper boundaries this collapses to putting all weight on a single grid point. We just need to do this before we evaluate the expectations term, and we will thus get $E[V_{j+1}(d,a,z')]$.

2.2.4 Finite Horizon: riskyasset

'riskyasset' is an endogenous state where aprime cannot be chosen directly, but is instead a function of a decision variable and a between-period i.i.d. shock; $aprime(d,u)$. The basic idea of how VFI Toolkit handles these is to evaluate $aprime(d_i,u_i)$ as a value on the grids of d and u, and then linearly interpolate this $aprime(d_i,u_i)$ onto the grid for a by allocating it to the grid-points above and below that value with linear weights.

So in a finite-horizon model with no other endogenous states, and no decision variables except those influencing the next period riskyasset, the algorithm is

Solve for final period, N_j: $V_{N_j}(a,z) = max_{d \in D(a,z)} F(d,a,z)$

... and keep the argmax g_{N_j}.
for j count backward from $N_j - 1$ to 1
Get θ_j from Θ
for A ll values z
Calculate $E[V_{j+1}(a', z')]$
Evaluate $aprime(d, u)$ on the grids for d and u
Interpolate $aprime(d, u)$ onto the grid for a; get $aprime_i(d, u)$ and $aProbs(d, u)$, the index for the lower grid point and the probability of the lower grid point.
Switch $E[V_{j+1}(a', z')]$ to $E[V_{j+1}(d, u, z')]$ using $aprime_i(d, u)$ and $aProbs(d, u)$ to replace a' with d and u.
Take expectations over u to convert $E[V_{j+1}(d, u, z')]$ to $E[V_{j+1}(d, z')]$
for A ll values of (a, z)
Calculate $V_j(a, z) = max_{d \in D(a,z)} F(d, a, z) + \beta E[V_{j+1}(d, z')]$
... and keep the $argmax$ $g_j(a, z)$.
end for
end for
end for
return $[V_1, V_2, ..., V_{N_j}]$, $[g_1, g_2, ..., g_{N_j}]$

notice that a' no longer appears in the return function. The main trick is to replace $E[V_{j+1}(a', z')]$ to $E[V_{j+1}(d, z')]$, by interpolating $aprime(d, u)$ onto the grid on a, and then taking expectations over u.

Note: An important improvement to runtimes in models with riskyasset is the use of 'refinement'. Essentially in many models some decision variables will appear in the ReturnFn but not the aprimeFn, and vice-versa. We can 'refine' out these decision variables before we combine the return function, $F(d, a, z)$ and the expectations term, $E[V_{j+1}]$. This massively shrinks the size of the combined problem and leads to both faster runtimes and lower memory use. It only makes sense to do on a gpu. Refine is explained for infinite horizon problems in Section 2.1.3. The linear interpolation is explained in Section 2.2.3.

2.2.5 Finite Horizon: standard & riskyasset

When using a standard endogenous state and a riskyasset together, the risky asset is always set up as the second endogenous state so we will call them $a1$ and $a2$, respectively. This approach is just to combine the usual approaches to each endogenous state, so the standard endogenous state will have $a1prime$ chosen on the grid while the riskyasset will be evaluated on the grids of d and u to get $a2prime(d_i, u_i)$ and then this is linearly interpolated onto the $a2$ grid by allocating it to the grid-points above and below that value with linear weights.

So in a finite-horizon model with no decision variables except those influencing the next period riskyasset, the algorithm is

Solve for final period, N_j: $V_{N_j}(a1, a2, z) = max_{d,a1' \in D(a1,a2,z)} F(d, a1', a1, a2, z)$

... and keep the argmax g_{N_j}.

for j count backward from $N_j - 1$ to 1

Get θ_j from Θ

for A ll values z

Calculate $E[V_{j+1}(a1', a2', z')]$

Evaluate $a2prime(d, u)$ on the grids for d and u

Interpolate $a2prime(d, u)$ onto the grid for $a2$; get $a2prime_i(d, u)$ and $a2Probs(d, u)$, the index for the lower grid point and the probability of the lower grid point.

Switch $E[V_{j+1}(a1', a2', z')]$ to $E[V_{j+1}(d, a1', u, z')]$ using $a2prime_i(d, u)$ and $a2Probs(d, u)$ to replace $a2'$ with d and u.

Take expectations over u to convert $E[V_{j+1}(d, a1', u, z')]$ to $E[V_{j+1}(d, a1', z')]$

for A ll values of $(a1, a2, z)$

Calculate $V_j(a1, a2, z) = max_{d,a1' \in D(a1,a2,z)} F(d, a1', a1, a2, z) + \beta E[V_{j+1}(d, a1', z')]$

... and keep the $argmax$ $g_j(a1, a2, z)$.

end for

end for

end for

return $[V_1, V_2, ..., V_{N_j}]$, $[g_1, g_2, ..., g_{N_j}]$

notice that $a2'$ does not appear in the return function. The main trick is to replace $E[V_{j+1}(a1', a2', z')]$ with $E[V_{j+1}(d, a1', z')]$, by interpolating $a2prime(d, u)$ onto the grid on $a2$, and then taking expectations over u.

2.2.6 Finite Horizon: Epstein-Zin preferences

There are various different ways to set up Epstein-Zin preferences, such as in consumption-units, in utility-units with positive utility, in utility-units with negative utility, and 'double Epstein-Zin' which treats the additional risk aversion differently for survival probabilities to other probabilities. VFI Toolkit handles all of these in a single code by taking advantage of the fact that we can set up a generic Epstein-Zin with eight 'Epstein-Zin coefficients' (called $ezc1$, $ezc2$,..., $ezc8$ in the codes), and then all the different Epstein-Zin preferences simply correspond to different values of these eight coefficients.[5]

The generic Epstein-Zin value fn problem for a finite-horizon value fn problem with markov

[5]This approach was developed from scratch for VFI Toolkit.

shocks is,

$$V_j(a,z) = \max_{d,a'} ezc1 * \big((ezc1 * F(d,a',a,z)^{ezc2}) + ezc3 * \beta * (s_j E[ezc4 * V_{j+1}(a',z')^{ezc5}|z]^{ezc8})^{ezc6}\big)^{ezc7}$$

The algorithm for solving this is,

Solve for final period, N_j: $V_{N_j}(a,z) = max_{d,a' \in D(a,z)}(ezc1 * F(d,a',a,z)^{ezc2})^{ezc7}$

... and keep the argmax g_{N_j}.

for j count backward from $N_j - 1$ to 1

Get θ_j from Θ

for A ll values z

Calculate $E[ezc4 * V_{j+1}(a',z')^{ezc5}|z]^{ezc8}$

for A ll values of (a,z)

Calculate $V_j(a,z) = max_{d,a' \in D(a,z)} ezc1 * \big((ezc1 * F(d,a',a,z)^{ezc2}) + ezc3 * \beta(s_j E[ezc4 * V_{j+1}(a',z')^{ezc}$

... and keep the $argmax$ $g_j(a,z)$.

end for

end for

end for

return $[V_1, V_2, ..., V_{N_j}]$, $[g_1, g_2, ..., g_{N_j}]$

Implementing the code for this requires a bit of care because there are issues around things like 0s and negative powers where you have to be careful to treat it correctly.

We now go through different Epstein-Zin setups and explain how the eight Epstein-Zin coefficients are set in each case. We use *ceis* and *crisk* to refer to the elasticity of intertemporal substitition and risk aversion parameters, respectively.

Traditional EZ in consumption units

$$\begin{aligned} ezc1 &= 1; \\ ezc2 &= 1 - 1./ceis; \\ ezc3 &= 1; \\ ezc4 &= 1; \\ ezc5 &= 1 - crisk; \\ ezc6 &= (1 - 1./ceis)./(1 - crisk); \\ ezc7 &= 1./(1 - 1./ceis); \\ ezc8 &= 1; \end{aligned}$$

EZ in utility-units (in utils): positive utility

$$
\begin{aligned}
ezc1 &= 1; \\
ezc2 &= 1; \\
ezc3 &= 1; \\
ezc4 &= 1; \\
ezc5 &= 1 - crisk; \\
ezc6 &= 1./(1 - crisk); \\
ezc7 &= 1; \\
ezc8 &= 1;
\end{aligned}
$$

EZ in utility-units (in utils): negative utility

$$
\begin{aligned}
ezc1 &= 1; \\
ezc2 &= 1; \\
ezc3 &= -1; \\
ezc4 &= -1; \\
ezc5 &= 1 + crisk; \\
ezc6 &= 1./(1 + crisk); \\
ezc7 &= 1; \\
ezc8 &= 1;
\end{aligned}
$$

Note: If the utility is negative use 1+crisk instead of 1-crisk. This way the interpretation of crisk is identical in both cases. And then we need to multiply it by -1 in two places, which is done by setting $ezc3$ and $ezc4$ to both be negative.

Double Epstein-Zin: inner EZ is about risk, out EZ is about mortality risk

$$\begin{aligned}
&\text{EZ in consumption units:}\\
ezc6 &= (1 - 1./ceis)./(1 - mrisk);\\
ezc8 &= (1 - mrisk)./(1 - crisk);\\
&\text{EZ in utils with positive utility:}\\
ezc6 &= 1./(1 - mrisk);\\
ezc8 &= (1 - mrisk)./(1 - crisk);\\
&\text{EZ in utils with negative utility:}\\
ezc6 &= 1./(1 + mrisk);\\
ezc8 &= (1 + mrisk)./(1 + crisk);
\end{aligned}$$

Note: this must be used together with one of the other settings, hence just showing which parameters need to change. $mrisk$ is the risk aversion parameter for the mortality risk.
Scale returns by $1-\beta$ or $1-s_j\beta$: Some people like to scale the return function by $1-\beta$ or $1-s_j\beta$ (done with vfoptions.EZoneminusbeta). This just requires setting $ezc1 = 1-\beta$ or $ezc1 = 1-s_j\beta$.

There is one aspect of Epstein-Zin preferences that cannot be accomodated by this approach, namely when the elasticity of intertemporal substitution is equal to one. As can clearly be seen in the definitions of the Epstein-Zin coefficients, this would lead to some of them being undefined due to division by zero, or blowing off to infinity. It is possible to solve Epstein-Zin preferences with the elasticity of intertemporal substitution is equal to one, but it cannot be fit into our generic Epstein-Zin problem. For how the problem has to be set up, see Sargent & Lungquist 4ed (2018) on pages 566-568, specifically looking at eqn (14.7.3) and compare the two cases. In practice, you could use the toolkit to solve with an elasticity of intertemporal substitition equal to 0.99 and again with 1.01, and as long as they give much the same solution, just interpret this as being a good enough approximation to the solution with 1.[6]

3 Agent Distribution

3.1 Stationary Agent Distribution in Infinite Horizon

There are three ways to compute the stationary agent distribution in infinite horizon models. The stationary agent definition is defined as $\mu(a,z)$ satisfying $\mu(a',z') = P(a',z'|a,z)\mu(a,z)$, where P is the markov transition kernel that combines the policy function $g(a'|a,z)$ with the exogenous shock transition kernel $\pi_z(z'|z)$ ($P = g \circ \pi_z$).

[6]Thanks to Louis Daumas for pointing me to where to find this.

The three ways to compute the stationary agent distribution are (i) as the eigenvector of $\mu = P\mu$, (ii) by simulation, and (iii) by iteration.

Which of the three to use? The iteration method is the default as it has a better combination of speed and accuracy than the simulation method. However the iteration method does require more memory and so for large models it is not an option; if you are getting out of memory errors when solving for the agent distribution then turn off iteration by setting *simoptions.iterate=0* and the toolkit will instead use the simulation. The eigenvector method is unstable so is never used.

3.1.1 Stationary Agent Distribution as Eigenvector

In principle this is simple, we know the stationary distribution is defined as satisfying $\mu = P\mu$, and once the problem is discretized P is a matrix. So μ is just an eigenvector of the matrix P and there are plenty of routines existing for finding eigenvectors in all standard software and they are very fast. In practice however the method is unstable as minor numerical rounding errors in computing the eigenvector mean it often contains negative elements. There are ways to 'bend' the eigenvector so that it is a probability distribution (all elements are positive, sum to one) as given, e.g., 'Step 5' on page 178 of Badshah, Beaumont, and Srivastava (2013) describes how to find the normalized eigenvector of the unit eigenvalue of P and use the Perron-Frobenius theorem to 'bend' it.

By default the toolkit will avoid the eigenvector approach due to the instability (if you want to use the eigenvector method, set *simoptions.eigenvector=1*). Neither of the ways to 'bend' it have been implemented (if you want to contribute one, please do).

3.1.2 Simulate Agent Distribution

We have a markov process P (on the joint space of the endogenous and exogenous states). We know from econometric theory that if we simulate a single time series using P then asymptotically the distribution of this time series will converge to what we are calling the stationary distribution $\mu = P\mu$.

Two minor points are needed to implement this. First, we have to start from somewhere, and we don't want the point we start from to matter, so we do 'burnin': simulate for B periods, throw these away except for where we end up, and now use this as the starting point for simulating our distribution (because the starting point is random it will not matter on average; it is also likely to be a 'typical' point). Second, simulating a single time series is slow, so in practice we simulate lots of time series in parallel on the cpu; while this is not in line with the underlying theory justifying the approach it works just fine in practice.

The pseudocode for simulating the agent distribution follows. Note everything is done based on the index of (a, z), not the value.

We will simulate M observations. Done as N simulations of length T ($M = N \times T$)
for i count from 1 to N (Parallel-for over cpu cores)
  Choose starting point (a_0, z_0)
  Create $Dist_i = zeros(size\ of\ (a,z)\ space)$
  for t counts from 1 to B ▷ B is number of periods for burnin
    Draw new (a_t, z_t) from $P(a', z'|a_{t-1}, z_{t-1})$
  end for
  for t counts from $B+1$ to T ▷ Same as during burnin, but now we will keep them
    Draw new (a_t, z_t) from $P(a', z'|a_{t-1}, z_{t-1})$
    $Dist_i(a_t, z_t) = Dist_i(a_t, z_t) + 1$ (recall, using indexes, not values)
  end for
  $Dist(:, i) = Dist_i$ ▷ Put the $Dist_i$ together
end for
$Dist = sum(Dist$,in the i dimension) ▷ $Dist$ is now a count of how many times each point in distribution was visited during the simulations
$Dist = Dist/sum(Dist)$ ▷ Normalize to be a probability distribution
return $Dist$

the implementation in VFI Toolkit uses the mid-point of the grids on (a, z) as the initial starting point (a_0, z_0) for all simulations (it will anyway be burned away). This can be controlled setting *simoptions.seedpoint* (which has default value of the mid-point of the grids on (a, z).

3.1.3 Iterate Agent Distribution

If we start from some initial guess for agent distribution μ_0, and iterate $\mu_t = P\mu_{t-1}$, we know from the theory that this sequence of μ_t will converge to the stationary distribution μ.[7]

Iteration just repeatedly iterates until convergence, so pseudocode is just
Start from initial guess μ_{old}
while currdist>tolerance
  $\mu = P\mu_{old}$ ▷ Iterate agent distribution
  $currdist = max(abs(\mu - \mu_{old}))$
  $\mu_{old} = \mu$ ▷ Update 'old'
end while
return μ ▷ The distribution once convergence was reached

This just leaves the question of what to use as the initial guess μ_{old}. By default VFI Toolkit just

[7] Typically P is either a 'stochastic contraction' or, more likely for incomplete markets models, satisfies a 'monotone mixing condition' (or 'splitting condition', or 'monotone order-mixing condition') and either of these means the sequence will converge. Actually analytically proving either of these is non-trivial but has been done for some of the more basic/standard heterogeneous agent incomplete market models.

put equal weight on every grid point as the initial guess for μ_{old}. You can give an initial guess for μ_{old} as *simoptions.initialdist* (if guess is good, codes will run faster).[8]

The code implementing this contains one trick to speed it up, namely that it computes $\mu = P\mu_{old}$ multiple times before calculating *currdist*; because the distance calculation is non-trivial it is costly to compute it every step. This is controlled by *simoptions.multiiter* which is set to 50 by default.

To avoid the risk of just iterating forever if the agent distribution is failing to converge, the codes impose a maximum number of iterations (it is an additional criterion of the while loop). This is controlled by *simoptions.maxit* and is set to 50,000 by default.

You can control 'tolerance' by setting *simoptions.tolerance*.

3.1.4 Tan Improvement to Iterating Agent Distribution

By default VFI Toolkit will perform the 'Tan improvement' Tan (2020) to iteration which we now describe. In standard iteration we use the transition matrix P, which is the composition of the policy function $g(a'|a,z)$ and the transition matrix for the exogenous state $\pi_z(z'|z)$; $P = g \circ \pi_z$. The Tan improvement is the clever observation that actually instead of doing the iteration as $\mu = P\mu_{old}$, we can actually do it as a two-step process, with the first step being $\tilde{\mu} = \Gamma\mu_{old}$ and the second step is $\mu = \pi_z\tilde{\mu}$; where Γ is just g but as a transition from (a,z) to (a',z) rather than just to a (notice that both P and Γ go from (a,z), but P goes to (a',z') while Γ goes to (a',z)). Conceptually this seems pointless, but computationally Γ is very *sparse* and so the Tan improvement is both vastly faster and uses less memory than standard iteration.

The pseudocode for iteration with the Tan improvement is thus,

Start from initial guess μ_{old}
Compute Γ from g
while currdist>tolerance
  $\mu = \Gamma\mu_{old}$ ▷ First step of Tan improvement
  $\mu = \pi_z\mu$ ▷ Second step of Tan improvement
  $currdist = max(abs(\mu - \mu_{old}))$
  $\mu_{old} = \mu$ ▷ Update 'old'
end while
return μ ▷ The distribution once convergence was reached

There is some reshaping of matrices involved in the Tan improvement that I have glossed over here, for a full explanation see Tan (2020). As in standard iteration our implementation does not check the *currdist* every iteration as it is faster not to.

[8] The toolkit used to use a small simulation as the initial guess, but the Tan improvement made the iteration so fast that getting a good initial guess was no longer worth the run time involved.

Because the Tan improvement makes the iteration step so much faster it is no longer worth generating a decent initial guess for μ_{old}, and so the default just puts all the mass for the endogenous state on the mid-point of the grid, and for the exogenous state it iterates ten times with π_z from putting equal mass on all grid points. You can give an initial guess for μ_{old} as *simoptions.initialdist* (if guess is good, codes will run faster).

3.2 Agent Distribution in Finite Horizon

There are two ways to compute the agent distribution in finite horizon, simulation and iteration. The user must define/input the period-1 distribution. The iteration method is used by default as it has a better combination of speed and accuracy than simulation, but because the iteration requires more memory it will sometimes not be possible and simulation should then be used instead (default is *vfoptions.iterate=1*, setting it to 0 means simulation will be used instead). When we use simulation we are essentially just doing a panel data simulation, but then converting the result into an agent distribution.

3.2.1 Simulating the Agent Distribution

The pseudocode for simulating the agent distribution follows. Note everything is done based on the index of (a, z), not the value. We use μ_j to denote the distribution at age/period j, and g denotes policy function, π_z denotes the transition matrix for the exogenous state.

Define initial age-1 distribution $\mu_1(a, z)$
Define age-weights $\omega(j)$
We will perform N simulations of length J
for i count from 1 to N (Parallel-for over cpu cores)
  Draw starting point (a_1, z_1) from $\mu_1(a, z)$
  Create $Dist_i = zeros(size\ of\ (a, z, j)\ space)$
  for j counts from 1 to J
    Get $a_j = g(a_{j-1}, z_{j-1}, j-1)$
    Draw z_j from $\pi_z(z_j|z_{j-1}; j)$
    $Dist_i(a_j, z_j, j) = Dist_i(a_j, z_j, j) + 1$ (recall, using indexes, not values)
  end for
end for
$Dist(:,:,:,i) = Dist_i$ ▷ Put the $Dist_i$ together
$Dist = sum(Dist$,in the i dimension) ▷ $Dist$ is now a count of how many times each point in distribution was visited during the simulations
$Dist(:,:,j) = Dist(:,:,j)/sum(Dist(:,:,j))$, for each j ▷ Normalize conditional on age to be a probability distribution
Multiply ω along the j dimension of $Dist$.

return $Dist$

3.2.2 Iterating the Agent Distribution

To iterate the agent distribution in finite horizon we need an initial distribution defined by the user, μ_1, and then construct the transition matrix on (a, z) for each period j denoted P_j. P_j is constructed by combining the policy function for period j, g_j, with the transition matrix on exogenous shocks for period j, $\pi_{z,j}$.

We just iterate j times,
Define initial age-1 distribution $\mu_1(a, z)$ (must be mass 1)
Define age-weights $\omega(j)$
for j counts from 1 to $J-1$
  Construct P_j from g_j and $\pi_{z,j}$
  $\mu_{j+1} = P_j \mu_j$ ▷ Iterate agent distribution
end for
Store all the μ_j in μ
Multiply ω along the j dimension of μ.
return μ

In the actual codes the storing of μ_j in μ is done inside the for-loop, and we just have a μ_{new} and μ_{old} rather than all of the μ_j.

VFI Toolkit by default will actually use the Tan improvement when iterating on the agent distribution and we turn to that now. We note that P here takes us directly from (a, z) to (a', z').

3.2.3 Tan Improvement Iterating the Agent Distribution

The Tan improvement is the observation that instead of using P we can use g and π_z one-by-one. This bit of genius means we do exactly the same, but in a way that is faster and less memory demanding (people spent 30+ years using P before Tan saw that you could split it in two!).

We just iterate j times,
Define initial age-1 distribution $\mu_1(a, z)$ (must be mass 1)
Define age-weights $\omega(j)$
for j counts from 1 to $J-1$
  Construct Γ_j from g_j
  $\mu_{temp} = \Gamma_j \mu_j$ ▷ First step of Tan improvement
  $\mu_{j+1} = \pi_{z,j} \mu_{temp}$ ▷ Second step of Tan improvement
end for

Store all the μ_j in μ
Multiply ω along the j dimension of μ.
return μ

There is some reshaping of matrices involved in the Tan improvement that I have glossed over here, for a full explanation see Tan (2020).

The Tan improvement can be understood as breaking the move from (a, z) to (a', z') into two steps. The first step is from (a, z) to (a', z), and the second step is from (a', z) to (a', z').

3.2.4 Tan Improvement with Semi-Exogenous shocks

Semi-exogenous shocks are exogenous states for which the transition probabilities depend on a decision variable (see Life-Cycle Model 30 for an example). So $\pi_{semiz}(semiz'|semiz, d)$ gives the probability of next-period semi-exogenous state based on the current semi-exogenous state and on the decision variable.

Using the Tan improvement requires a minor modification. Note that we cannot treat $semiz$ like we treat z, in the second step of the Tan improvement, because semiz depends on d (and hence on a and z). Instead we have to include $semiz$ in the first step of the Tan improvement.

We just iterate j times,
Define initial age-1 distribution $\mu_1(a, semiz, z)$ (must be mass 1)
Define age-weights $\omega(j)$
for j counts from 1 to $J-1$
  Construct Γ_j from g_j and $\pi_{semiz,j}$
  $\mu_{temp} = \Gamma_j \mu_j$ ▷ First step of Tan improvement
  $\mu_{j+1} = \pi_{z,j} \mu_{temp}$ ▷ Second step of Tan improvement
end for
Store all the μ_j in μ
Multiply ω along the j dimension of μ.
return μ

The difference from the standard Tan improvement is all in the first step, which now includes the semi-exogenous state transitions alongside the standard endogenous state transitions. There is some reshaping of matrices involved in the Tan improvement that I have glossed over here, for a full explanation see Tan (2020). Note that now our distribution is on $(a, semiz, z)$, and Γ_j is used in the first step to change to $(a', semiz', z)$, and then the second step goes to $(a', semiz', z')$.[9]

[9]Getting the Tan improvement to work here does require the order $(a, semiz, z)$, in fact the whole toolkit got rewritten just to be able to do this as it originally had z then $semiz$.

3.2.5 Iterating the Agent Distribution with 'nProbs'

Sometimes our policy for the next-period endogenous state does not sit on a single grid point. It might be 'between' two grid points (e.g., experienceasset, grid interpolation layer, or residualasset), or it might send us to several grid points at once (e.g., riskyasset or experienceassetu, where each value of an i.i.d. shock u leads to a different between-grid-points value). VFI Toolkit handles all of these with a single unified approach called 'nProbs'.

The idea of nProbs is that we represent the policy for the next-period endogenous state as N_probs next-period grid points together with N_probs probabilities. The probabilities sum to one, because the agent goes somewhere next period with certainty. Once we have the N_probs grid points and their probabilities, the rest of the agent distribution iteration is unchanged. Several familiar cases are just particular values of N_probs:

- A standard on-grid policy is $N_probs = 1$: a single next-period grid point with probability one. [Just pointing out the relation, these do not use the $nProbs$ codes, just the standard codes.]
- A 'between two grid points' policy is $N_probs = 2$: the lower and upper grid points, with two probabilities. This covers experienceasset, grid interpolation layer, and residualasset.
- A policy driven by an i.i.d. shock is $N_probs = 2 * n_u$ (this is what was previously called 'uProbs'): for each of the n_u values of u we get a between-grid-points policy, i.e. a lower and upper grid point. This covers riskyasset and experienceassetu.

More elaborate combinations simply give larger N_probs; for example a standard endogenous state that is done with grid interpolation layer used together with a riskyasset gives $N_probs = 2 * n_u * 2$ (lower/upper for the riskyasset, for each u, crossed with lower/upper for the standard asset with grid interpolation layer).

We store the policy as the N_probs next-period grid point indices $Policy_l(a, z, \cdot, j)$ and the corresponding probabilities $Policy_\pi(a, z, \cdot, j)$, where the extra dimension has length N_probs and the probabilities sum to one over that dimension.

Getting the grid points and probabilities: the between-grid-points case. Consider a single between-grid-points policy value $a*$ (we will have one for each point in the state space, so we do this many times; here just consider one). Our grid of values on the endogenous state is $a_grid = \{a_1, a_2, \dots, a_{n_a}\}$. We first find the lower grid point, the grid point immediately below $a*$, by solving $\arg\max_{i=1,\dots,n_a}\{a_i \leq a*\}$; call it a_{i_l}. By definition the upper grid point is one above, a_{i_l+1}. The probability of the lower grid point is $\pi_l = 1 - \frac{a*-a_{i_l}}{a_{i_l+1}-a_{i_l}}$; this is linear interpolation, so if $a*$ is one-quarter of the way from the lower to the upper grid point then the lower grid point gets

probability three-quarters, and if it is halfway it gets one-half. The upper grid point probability is $1-\pi_l$. This is the $N_probs = 2$ case: two grid points (a_{i_l}, a_{i_l+1}) with probabilities $(\pi_l, 1-\pi_l)$.

experienceasset and residualasset produce the policy as such an $a*$ value directly. For grid interpolation layer the *Policy* instead already stores the lower grid point on a_grid (so we have the lower grid point directly) together with the 'interpolation-layer index', which is $n2$ points evenly spaced between two consecutive grid points of a_grid (the index runs from 1 to $n2+2$, as it includes the two endpoints). Since these interpolation-layer points are evenly spaced, the lower grid point probability is simply $\pi_l = 1 - (\text{interpolation-layer-index}-1)/(n2+1)$. Either way we end up with the same two grid points and two probabilities.[10]

Getting the grid points and probabilities: the i.i.d.-shock case. For riskyasset and experienceassetu, each of the n_u values of the i.i.d. shock u gives a different between-grid-points value. We apply the between-grid-points construction above to each of the n_u values, giving $2 * n_u$ grid points. The probability attached to each of these grid points is the interpolation weight (π_l or $1-\pi_l$ for that u) multiplied by the probability of that value of u (from π_u). So again we have N_probs grid points with N_probs probabilities that sum to one, just now with $N_probs = 2 * n_u$.

Constructing the transition and iterating. Once the policy is in the form of N_probs grid points and probabilities, constructing the transition is a small change from the on-grid case. First consider the simpler setting with no markov z, just an endogenous state a. With an on-grid policy, the transition matrix P_j from (a) to (a') had a single 1 in each row. With an nProbs policy, each row instead has N_probs non-zeros, in the columns of the N_probs next-period grid points, with values equal to their N_probs probabilities. Putting the markov z back in, the on-grid P_j had n_z non-zeros per row (the single a' crossed with the n_z next-period z', with probabilities from π_z), while the nProbs P_j has $N_probs * n_z$ non-zeros per row, with probabilities given by $\pi_z(z_c, :)$ crossed with the N_probs policy probabilities.

Once we have P_j for $j = 1, \ldots, J-1$, the rest of the agent distribution algorithm is exactly as before; the iteration

Define initial age-1 distribution $\mu_1(a, z)$ (must be mass 1)
Define age-weights $\omega(j)$
for j counts from 1 to $J-1$
  Construct P_j from g_j (now as N_probs grid points and probabilities) and $\pi_{z,j}$
  $\mu_{j+1} = P_j \mu_j$ ▷ Iterate agent distribution
end for
Store all the μ_j in μ

[10] The use of 'nProbs' does not depend on linear interpolation. In VFI Toolkit it is (currently) always linear interpolation that is used to set the probabilities, but in principle any other scheme could be used.

Multiply ω along the j dimension of μ.
return μ

is unchanged except that P_j now has more non-zeros.[11] In practice VFI Toolkit does not build P_j directly: it uses the Tan improvement, constructing Γ_j from the N_probs grid points and probabilities (as a sparse matrix, letting repeated indices accumulate) for the first step, and then applying $\pi_{z,j}$ for the second step, exactly as in the on-grid Tan improvement described earlier. This is how `StationaryDist_FHorz_Iteration_nProbs_raw.m` is organized.

4 Stationary General Equilibrium

To solve for a stationary general equilibrium, we are looking for some parameters that are determined in general equilibrium, call them θ such that our general equilibrium equations equal zero, $GEeqn(\theta) = 0$.

There are various algorithms for doing this (*heteroagentoptions.fminalgo* determines which is used), but all of them essentially boil down to the following,

Guess some initial θ^0
Define $GeneralEqmConditions = \infty$
while $GeneralEqmConditions > tolerance$
  Solve the value fn and policy (given current θ^k)
  Solve the agent distribution (given current θ^k and policy we just found)
  Solve the aggregate variables at each t (given current θ^k and policy/agent dist we just found)
  Evaluate $GeneralEqmConditions = GEeqn(AggVars, \theta^k)$ (given current θ^k and agg vars)
  Update θ^{k+1} (in some way, likely based on θ^k and $GeneralEqmConditions$)
end while
return θ

The difference between the different *fminalgo* is how we update the θ. Note that in almost all of the *fminalgo* the while loop is being done by an optimization algorithm (like Matlab's *fminsearch()*).

Whether the problem is finite or infinite-horizon value functions, and whether or not there are permanent types of different kinds of shocks is unimportant, in the sense that the above steps are all the same steps, just that the details of their implementation is changed.

When there are multiple general equilibrium equations, $GeneralEqmConditions$ is (by default) defined as the sum-of-squares of the individual general equilibrium equations.

[11] Only the non-zeros matter in practice as you will always want to implement this using sparse matrices.

5 Transition Path General Equilibrium

We consider a transition path of length T time periods. Solving for a transition path general equilibrium, we are looking for some path-of-parameters that are determined in general equilibrium, call them Θ such that our general equilibrium equations equal zero in every time period, $GEeqn(\Theta) = 0$. Note that at some level this is the same thing we did for a stationary equilibrium, except that now $\Theta = \{\theta_1, ..., \theta_T\}$ is a time-path of general-eqm parameters, rather than just parameters at a single point in time.

There are various algorithms for doing this (*heteroagentoptions.fminalgo* determines which is used), but all of them essentially boil down to the following,

Guess some initial Θ^0 (time-path on general-equilibrium parameters)
Define $GeneralEqmConditions = \infty$
while $GeneralEqmConditions > tolerance$
Solve backward, $t = T-1, \ldots, 2, 1$ for the value fn and policy at each t (given current θ_t^k)
Solve forward, $t = 1, 2, \ldots, T$ for the agent distribution (given current θ_t^k and $Policy_t$ we just found)
Solve the aggregate variables (given current θ_t^k and policy/agent dist we just found for t)
Evaluate $GeneralEqmConditions = [GEeqn_1(AggVars_1, \theta_1^k),\ GEeqn_2(AggVars_2, \theta_2^k), \ldots, GEeqn_T(AggVars_T, \theta_T^k)]$ (evaluate general eqm eqns at each time period)
Update Θ^{k+1} (in some way, likely based on Θ^k and $GeneralEqmConditions$)
end while
return Θ

The difference between the different *fminalgo* is how we update the Θ. The most common way to update is as a shooting algorithm.

In a stationary equilibrium, parameters other than those parameters which were being determined in general equilibrium (the general equilibrium parameters) were all constant (in time) and kept in Parameters structure. When we solve transition paths, some parameters might vary over time, and this is done with the ParamPath (a path on exogenous parameters), while any other parameters not in this path are assumed to be constant at their values in Parameters structure. It is not described in algorithm above, but obviously we have to make sure to use the correct parameters in each time period.

It is important to solve backward for policy and forward for agent distribution. For convenience, aggregate variables and general eqm conditions are often solved forwards while doing agent distribution, but in principle how we deal with these (in terms of timing) is not important in most models (can be important if we use lagged/leading parameter values for something).

Whether the problem is finite or infinite-horizon value functions, and whether or not there are

permanent types of different kinds of shocks is unimportant, in the sense that the above steps are all the same steps, just that the details of their implementation is changed.

References

Muffasir Badshah, Paul Beaumont, and Anuj Srivastava. Computing equilibrium wealth distributions in models with heterogeneous-agents, incomplete markets and idiosyncratic risk. Computational Economics, 41, 2013. doi: https://doi.org/10.1007/s10614-011-9313-8.

Ivo Bakota and Matthias Kredler. Continuous-time speed for discrete-time models: A markov-chain approximation method. MEA Discussion Papers, 2022. doi: https://dx.doi.org/10.2139/ssrn.4155499.

Robert Bray. Markov decision processes with exogenous variables. Management Science, 2017. doi: https://doi.org/10.1287/mnsc.2018.3158.

Robert Bray. Strong convergence and dynamic economic models. Quantitative Economics, 10, 2019. doi: https://doi.org/10.3982/QE833.

Robert Kirkby. VFI toolkit, v2. Zenodo, 2022. doi: https://doi.org/10.5281/zenodo.8136790.

Thomas Phelan and Keyvan Eslami. Applications of markov chain approximation methods to optimal control problems in economics. Journal of Economic Dynamics and Control, 2022. doi: https://doi.org/10.1016/j.jedc.2022.104437.

Pontus Rendahl. Continuous vs. discrete time: Some computational insights. Journal of Economic Dynamics and Control, 2022. doi: https://doi.org/10.1016/j.jedc.2022.104522.

Eugene Tan. A fast and low computational memory algorithm for non-stochastic simulations in heterogeneous agent models. Economics Letters, 193, 2020. doi: https://doi.org/10.1016/j.econlet.2020.109285.